Azure Deployment Design

Codebase Q&A and PR Review Assistant, Phase 5

Author	Todd Oppel
Repository	github.com/tpotechno/codebase-qa-assistant
Version	0.6.2 (Phase 5 deployment complete)
Date	November 2, 2026
Status	Merged in PRs #3–#7; live on Azure; 67 server tests and 13 Angular tests passing

PDF version: azure_design.pdf · Setup guide: azure_deployment.md · Earlier phases: mcp_server_design.md, pr_review_design.md, web_ui_design.md

1. Summary

Phase 5 takes the assistant from a laptop to the cloud. Two apps run on **Azure Container Apps** from one container image:

- **A public demo.** The web UI with a scripted model over the project's own source. Anyone can try it, and it costs nothing in model usage, because there is no API key anywhere in it.
- **A private app.** The real assistant: Claude answers questions, the Agent SDK reviews real GitHub pull requests, and reviews can be posted. Sign-in is with GitHub, and only allowlisted users get through.

Every merge to `main` builds the image, pushes it to Azure Container Registry, and rolls out both apps from GitHub Actions. The workflow signs in to Azure with **OpenID Connect**, so no Azure credential is stored in GitHub. All infrastructure is described in **Bicep**. The private app's keys live in **Key Vault**, and the app holds only references to them.

2. Why

A portfolio project is judged in minutes. A link that opens a working app beats a README that asks someone to clone, install and add API keys. Hosting it also exercises what production work involves that a local project never touches: identity, secrets, infrastructure as code, deploy pipelines, and the security of a server that spends money on its owner's behalf.

The private app makes the assistant useful from any browser, with nothing to install.

Goals

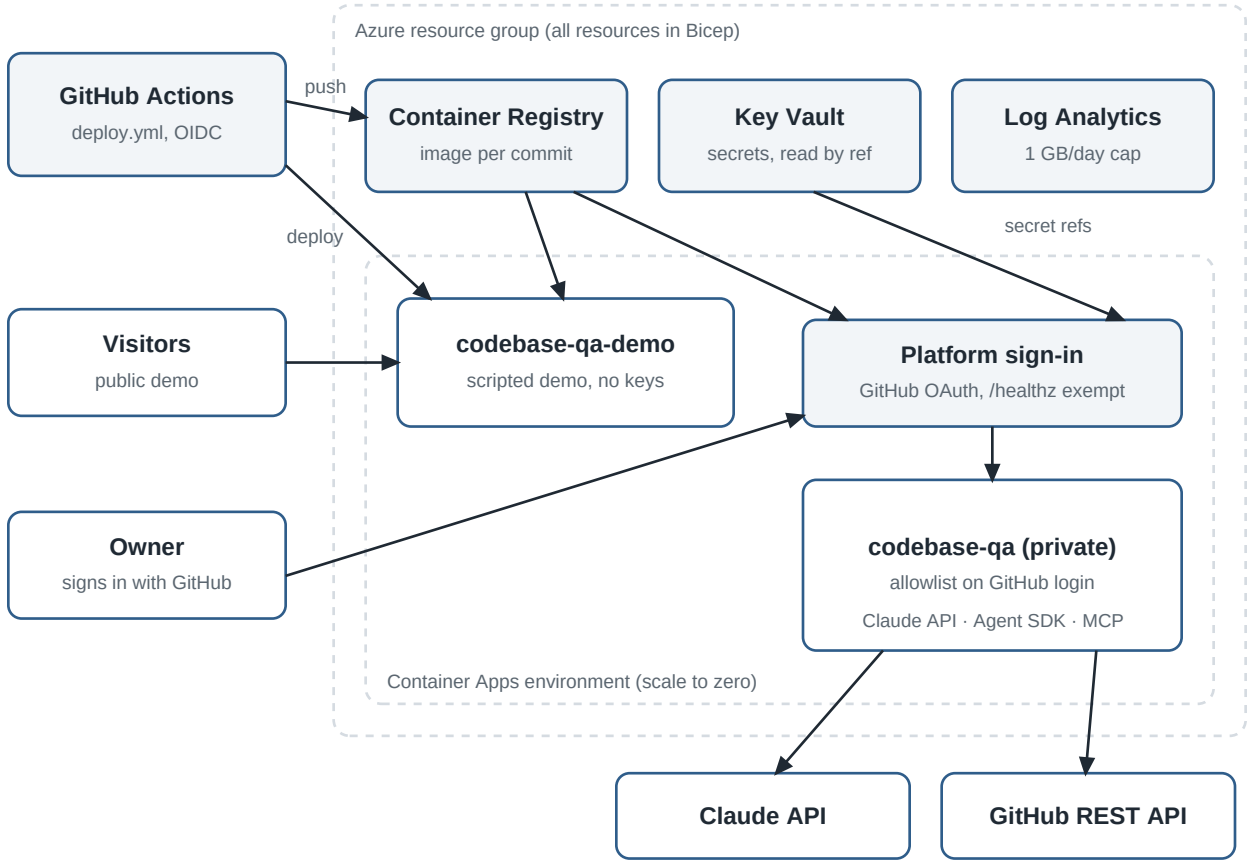
- A public URL that shows the product working, at no model cost and with no abuse risk.
- The real assistant hosted for its owner only, with credentials no one else can use.
- No long-lived secrets in GitHub, in the image, or in the Bicep.
- Every Azure resource defined in code and reviewed in pull requests.
- Deploys on merge, with automated checks that the live apps are healthy and locked down.
- Close to zero cost while idle.

Non-goals (this phase)

- Multiple users with their own keys and repositories (the private app serves one owner).
- Asking about repositories other than the one built into the image.
- Horizontal scaling: one replica per app, because drafts and the run limit live in memory.
- A custom domain.

3. What was built

Architecture



Arrows from the registry are image pulls by each app's managed identity; no registry password exists.

Figure 1. Runtime architecture. Dashed outlines group what runs together.

Figure 1 in the PDF draws this. A merge to `main` triggers `deploy.yml`, which signs in to Azure with a short-lived OIDC token. It deploys `registry.bicep`, builds and pushes the image tagged with the commit SHA, then deploys `main.bicep` (the demo) and `private.bicep` (the private app). Both apps pull the image with managed identities. The private app also reads its secrets from Key Vault with its identity. Visitors to the private app pass through Container Apps authentication, which signs them in with GitHub, before any request reaches the server.

Components

File	What it is
`Dockerfile`	Two stages: build the Angular UI, then a Node 24 runtime as the unprivileged `node` user. The repository source is copied in as the corpus the tools read.
`infra/registry.bicep`	Container Registry (Basic, admin user off). Grants `AcrPush` to whoever deploys it, through `deployer()`.
`infra/main.bicep`	Log Analytics (1 GB/day cap), the Container Apps environment, a pull identity with `AcrPull`, and the demo app (0.25 vCPU, scales 0–1, `/healthz` probes).
`infra/private.bicep`	Key Vault (RBAC), the private app's identity (`AcrPull` + `Key Vault Secrets User`), and, once enabled, the private app with Key Vault secret references and the GitHub sign-in configuration.
`.github/workflows/deploy.yml`	OIDC sign-in, build and push, three Bicep deployments, smoke tests for both apps.
`.github/workflows/ci.yml`	Adds a job that lints and builds the Bicep, builds the image, runs it, and checks health, demo mode, the Host guard, and that the real server refuses to listen publicly without an allowlist.
`src/web-server.ts`	`WEB_HOST`, `PUBLIC_HOSTS` and `ALLOWED_GITHUB_USERS` for hosting. Refuses to bind to a non-loopback address without an allowlist.
`src/web-app.ts`	`/healthz` ahead of the Host check, security headers on every response, and the sign-in gate (`requireUser`) that reads the GitHub login from the platform's principal.
`scripts/demo-server.ts`	`DEMO_HOST` and `PUBLIC_HOSTS` for the container, more concurrent runs for shared use, a bounded request log.
`web/`	A "Scripted demo" notice, a suggestion and sample-PR button that match the script, the signed-in user with a sign-out link, and a hosted-aware error message.

One image, two apps

The demo and the private app run the same image. The demo uses the image's default command (`scripts/demo-server.ts`); the private app overrides it with `src/web-server.ts`. So each merge is built once and the two apps never drift apart. CI checks the image the same way for both: the demo must answer correctly, and the real server must refuse to start publicly without an allowlist.

4. How it works

Deploying

1. `azure/login` exchanges GitHub's OIDC token for an Azure token. The federated credential trusts one subject: this repository's `azure-demo` environment, identified by the owner's and repository's numeric IDs.
2. `registry.bicep` makes sure the registry exists (a no-op after the first run).
3. The image is built and pushed, tagged with the commit SHA. Sign-in and push retry briefly, because a new role assignment can take a minute to take effect.
4. `main.bicep` rolls out the demo. Container Apps starts a new revision and moves traffic once `/healthz` passes.

5. The smoke test wakes the demo if it scaled to zero, then checks `/healthz`, that `/api/config` reports demo mode, and that the UI is served.
6. `private.bicep` deploys in two stages (below). Once enabled, a second check confirms the private app is healthy **and** that an unauthenticated `/api/config` gets a redirect or 401, never data.

Bringing up the private app in two stages

The app can't start until its secrets exist, and the GitHub OAuth app can't be registered until the app's URL is known. So `private.bicep` takes a `deployApp` flag:

- **Stage 1** (every deploy): the Key Vault and the app's identity. The run summary prints the vault name, the app's future URL, and the OAuth callback URL. The host name is predictable (`<app>.<environment's default domain>`).
- **Stage 2** (`PRIVATE_APP_ENABLED=true`): the Container App and its authentication configuration. This happens once the owner has registered the OAuth app and put three secrets in the vault.

Signing in

1. An unauthenticated visitor is redirected to GitHub by Container Apps authentication. Only `/healthz` is exempt, for the platform's probes.
2. After sign-in, the platform attaches the identity to every request in `X-MS-CLIENT-PRINCIPAL` (base64 JSON with the provider and its claims), overwriting any copy a client sends.
3. `requireUser` takes the GitHub **login** from those claims: `urn:github:login`, otherwise the API or profile URL claims, which GitHub derives from the login. User-editable claims (display name, blog) are never used. Only logins in `ALLOWED_GITHUB_USERS` are admitted. Anyone else gets a 403.
4. `/api/config` returns the login, and the UI shows it with a sign-out link.

5. Security

- **No stored Azure credential.** GitHub Actions authenticates with OIDC. The federated credential's subject includes the repository's numeric ID, so a fork, another branch's workflow, or a future repository with the same name can't match it. The deploy identity's roles are scoped to one resource group.
- **No secret in GitHub, the image or the Bicep.** The Anthropic key, the GitHub token and the OAuth client secret are written to Key Vault by the owner from Cloud Shell (`read -s` keeps them off screen and out of history). The Container App holds Key Vault references read by its own identity. Repository variables hold only identifiers.
- **No registry passwords.** Pushes use the deploy identity (`AcrPush`); pulls use managed identities (`AcrPull`).
- **Defense in depth on the private app.**
  - Platform sign-in at the door.
  - The allowlist in the app, matched on the immutable login.
  - The server refuses to listen publicly without an allowlist, which CI tests against the real image.

  - The Host and Origin checks from Phase 4, plus same-origin JSON for every state-changing request.
- **The public demo can't spend money.** It has no key, and its tool calls are scripted, so visitors can't steer them.
- **Headers on every response:** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: same-origin`.
- **Spend limits carry over:** the per-review dollar cap, the 10-minute timeout, two concurrent runs, and the read-only review agent.

6. Key decisions and alternatives

Decision	Chosen	Alternatives considered
Hosting	Azure Container Apps	App Service (no scale to zero on cheap tiers); AKS (far more than needed); Static Web Apps + Functions (streaming and the Agent SDK's child process fit containers better)
Deploy auth	GitHub OIDC federated credential	A service principal secret or publish profile stored in GitHub (long-lived, must be rotated)
Infrastructure	Bicep, deployed by the workflow	Portal clicks (not reviewable or repeatable); Terraform (an extra tool and state file for one resource group)
Registry	ACR Basic with managed identities	GitHub Container Registry (free, but pulling a private image needs a stored token)
Private-app sign-in	Container Apps authentication with GitHub, plus an in-app allowlist	Microsoft Entra ID (fits less naturally for a GitHub tool); sign-in written into the app (more code to get wrong)
Secrets	Key Vault references	Container App secrets set by the pipeline (the values would have to pass through GitHub)
Demo model	Scripted, no key	The real model with a tiny budget (still abusable and still costs money)
Images	One image, two commands	Separate images (double the build, and they can drift)

7. Deployment history

Getting the deploy right meant clearing several problems in a real Azure subscription. Each fix went into the setup guide or the code, so they don't recur.

Problem	Cause	Fix
Azure sign-in failed with `AADSTS700213`	GitHub now issues OIDC subjects with numeric IDs (`repo:owner@id/repo@id:environment:...`); the documented subject had none	The setup builds the exact subject from the repository's IDs; troubleshooting explains how to compare (PR #4)
Retried sign-in still failed	A new federated credential takes minutes to take effect	Waited, then re-ran; noted in troubleshooting

Problem	Cause	Fix
Demo deploy failed with `MissingSubscriptionRegistration`	`Microsoft.ManagedIdentity` wasn't registered on the subscription	Added to the setup's provider list (PR #4)
Node 20 deprecation warnings	Action versions pinned to Node 20 releases	`checkout` v7, `setup-node` v7, `azure/login` v3, inputs verified unchanged (PR #4)
GitHub callback returned 401	The OAuth client secret in Key Vault was wrong (a stray character from copying)	Regenerated the secret, stored it again, restarted the revision
Signed in but refused: "Todd Oppel isn't allowed"	The platform puts the **display name** in `X-MS-CLIENT-PRINCIPAL-NAME`, not the login. Matching it would also have been a hole: anyone can set their display name to `tpotechno`	Match the login from the principal's claims only, with tests for an impostor display name (PR #6)
Every Ask and Review got an empty 403	The sign-in layer rejects signed-in POSTs that carry no `Referer`, and the app sent `Referrer-Policy: no-referrer`	Diagnosed live: GETs passed, all POSTs failed at the middleware, CORS made no difference, and the same POST with a same-origin referrer reached the app. Switched to `same-origin` (PR #7)

Two of these were found with nothing but the browser's developer tools, by reading response headers. The app stamps every response with its own security headers. A response without them, carrying `x-ms-middleware-request-id`, had been refused by the platform before reaching the app. That one observation separated the app's problems from the platform's.

8. Testing

- **Server tests** (67): the sign-in gate covers:
  - a missing identity (401);
  - another user (403);
  - the right login from the wrong provider (403);
  - the owner admitted by login with a different display name;
  - the login taken from the URL claim;
  - an impostor whose display name is the owner's username (403);
  - user-editable claims ignored;
  - a malformed principal and a malformed login;
  - `/healthz` open while the API stays gated.

Also: `/healthz` answering any Host, the security headers including the referrer policy, and the demo flag.

- **Angular tests** (13), unchanged, plus a production build.
- **CI image job:**
  - Bicep lint and build;

  - `docker build`;
  - a running container checked for health, demo mode, and the Host guard (403 for a foreign Host);
  - the real server refusing to start on `0.0.0.0` without an allowlist.
- **Deploy smoke tests** against the live apps, on every merge.
- **Manual checks on the live private app:** sign-in, Ask, PR review, and posting a review.

9. Configuration and usage

The step-by-step setup is in azure_deployment.md. In short:

Where	Name	Purpose
Repository variables	`AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `AZURE_RESOURCE_GROUP`	Where and as whom the workflow deploys
Repository variables	`PRIVATE_APP_ENABLED`, `PRIVATE_OAUTH_CLIENT_ID`, `PRIVATE_ALLOWED_USERS` (optional)	Turn on the private app and its sign-in
Key Vault secrets	`anthropic-api-key`, `github-token`, `github-oauth-secret`	The private app's credentials
Server environment	`WEB_HOST`, `PUBLIC_HOSTS`, `ALLOWED_GITHUB_USERS`, `DEMO_HOST`	Set by the Bicep; unset for local use

Cost: about $5 a month, nearly all of it the registry. Both apps scale to zero, so idle compute falls within the Container Apps free grant. Key Vault costs fractions of a cent. The private app's real cost is Claude usage, capped per review.

10. Limitations and next steps

- **One repository.** The private app answers questions about the source built into its image. Next: clone a chosen repository at startup, or on demand per question.
- **One replica.** Drafts live in memory. Next: a small store (Table Storage or Redis) would allow more replicas and survive restarts.
- **Cold starts.** The first request after an idle period waits a few seconds while a replica starts.
- **Evaluation.** An eval suite for answer grounding and review quality, run in CI, remains on the roadmap.
- **A hosted MCP endpoint.** The MCP server could be its own Container App behind a bearer token, so any MCP client could use it remotely.
- **Azure DevOps pull requests**, behind the same interface as GitHub.